

For even n these patterns are solution grids to a sudoku-like logic puzzle usually called Binairo or Takuzu.

Tests with $n = 1$ to $n = 4$ are unsolvable, everything else does have a solution.

Small test cases are solvable via brute force.

Medium-sized tests are solvable by reducing them to an ILP and using a solver for that. (Use one binary variable for each cell. Constraints on the correct numbers of 0s in rows and columns and constraints on no three equals in a row are simple constraints on sums of these variables. The tricky bit is modeling “no two lines are equal”, which can be modeled as follows: For each pair of lines we’ll have n new variables representing absolute differences between corresponding elements (or, in other words, their xor). The value of each “xor-variable” can be enforced by adding four simple linear inequalities. And once we have the xors, we can add one final constraint that their sum must be positive.

Another way of dealing with this constraint is to add some random noise (either to the starting state or by telling the solver to randomize its search, if it supports such a thing) and just checking whether the output we got works.

Medium-sized tests can also be solved by local optimizations: define a badness function that measures how far your input is from what's required and then run any standard optimization such as hill-climbing.

In order to find a full solution one helpful step is to realize that a simple checkerboard already satisfies almost all the constraints, and we “just” need to tweak it locally to make all rows and columns distinct.

One way to do this is to try starting the previously mentioned local optimizations not from a random state but from the “already almost solved” checkerboard pattern.

Another way is to actually find a systematic pattern of changes by playing around with the checkerboard patterns on paper.

One easy-to-implement pattern that works is shown below:

0011010101010101010101010101
1100101010101010101010101010
0100110101010101010101010101
1011000101010101010101010101
0101001101010101010101010101
1010110010101010101010101010
0101010011010101010101010101
1010101100101010101010101010
010101001110101010101010101
1010101110010101010101010101
0101010100110101010101010101
1010101011100101010101010101
0101010101001101010101010101
10101010101010110010101010101
0101010101010110010101010101
101010101010101010011010101
0101010101010101010011010101
10101010101010101010110010101
01010101010101010101001101
101010101010101010101100101
01010101010101010101010011
101010101010101010101010100
001010101010101010101010101
001010101010101010101010101

Note that this pattern creates boxes of 2×2 zeros by modifying the main diagonal + two adjacent ones. Once we create that pattern, we are left with just a constant number of issues, and based on the parity of n those can be solved by flipping a constant number of bits in the bottom right or bottom left corner of the grid.